Query Redis from PostgreSQL in Supabase Using Foreign Data Wrappers (FDW):

Setting up **PostgreSQL to communicate with Redis** using a **Foreign Data Wrapper (FDW)**. This allows us to **query Redis like a normal table in PostgreSQL**.

Benefits of Accessing Redis as a Table in PostgreSQL:

1️⃣ **Query Redis with SQL:** Use SQL to access Redis data—no need to learn Redis commands.

2️⃣ **Join with PostgreSQL Tables:** Combine Redis and PostgreSQL data in a single SQL query.

3️⃣ **Use SQL Features:** Apply filters, sorting, and aggregation on Redis data using PostgreSQL.

4️⃣ **Reduce API Calls:** Fewer Redis API calls mean faster performance and less network overhead.

5️⃣ **Centralized Access:** Apps only connect to PostgreSQL, simplifying your architecture.

Steps to query redis from postgres supabase:

1. Enable Wrappers
2. Enable the Redis
3. WrapperStore your credentials (optional)
4. Connecting to Redis
5. Create a schema
6. Create foreign tables
7. Query redis

Step by Step Guide:

---

1. **Enable Wrappers:**

   CREATE EXTENSION IF NOT EXISTS wrappers WITH SCHEMA extensions;

Explanation:

1. **CREATE EXTENSION** – This command is used to install an extension in PostgreSQL. **Extensions add extra functionality to the database.**

2. **IF NOT EXISTS** – This ensures that the extension is only created if it does not already exist. If the extension is already installed, the command does nothing and avoids errors.

3. **wrappers** – This is the name of the extension being installed. The wrappers extension is used in PostgreSQL **to provide Foreign Data Wrappers (FDWs) and other utilities** for connecting to external data sources.

4. **WITH SCHEMA extensions** – This specifies that the extension should be installed into the **extensions** schema instead of the default public schema.

Why Use This?

- Ensures the extension is installed without duplicating it.
- Keeps the extension within a dedicated schema (extensions), avoiding conflicts with other database objects.

command to check the schemas:

SELECT schema_name FROM information_schema.schemata;

2. Enable the Redis

```
CREATE FOREIGN DATA WRAPPER redis_wrapper

HANDLER redis_fdw_handler

VALIDATOR redis_fdw_validator;
```

The Redis Wrapper allows you to read data from Redis within your Postgres database like it's a table.

PostgreSQL normally stores data inside itself.
But sometimes, we want to get data from **another database** (like Redis).

A Foreign Data Wrapper (FDW) is like a bridge.
It lets PostgreSQL connect to **other databases** and use them as if they were part of PostgreSQL.

This command creates a bridge (FDW) for Redis.

- redis_wrapper → The name of the bridge.
- redis_fdw_handler → The function that **handles communication** between PostgreSQL and Redis.
- redis_fdw_validator → The function that **checks for mistakes** when setting up this connection.

Why Use This?

- You can query **Redis data directly from PostgreSQL**.
- It's useful for **caching** and **reducing database load**.

3. WrapperStore your credentials (optional):

```
-- Save your Redis connection URL in Vault and retrieve the `key_id`
INSERT INTO vault.secrets (name, secret)
VALUES (
  'redis_conn_url',
  'redis://username:password@127.0.0.1:6379/db'
)
RETURNING key_id;
```

This SQL **inserts** a Redis connection URL into a database table (vault.secrets) and **returns the key_id** of the inserted record.

INSERT INTO vault.secrets (name, secret)

- This inserts data into the vault.secrets table.
- vault is likely a schema that organizes security-related tables.
- secrets is the table where sensitive information (like credentials) is stored.

VALUES ('redis_conn_url', 'redis://username:password@127.0.0.1:6379/db')

- The **first value (**'redis_conn_url'**)** is the **name of the secret**.
- The **second value** is the **actual Redis connection URL**, which includes:
    - redis:// → Protocol
    - username:password@ → Redis authentication
    - 127.0.0.1:6379 → Redis server IP and port
    - /db → The Redis database number

RETURNING key_id;

- After inserting the data, this **returns the key_id** of the newly inserted secret.
- key_id is likely a **unique identifier** (like a primary key) for each stored secret.

---

4. Connecting to Redis

```
CREATE SERVER redis_server
  FOREIGN DATA WRAPPER redis_wrapper
  OPTIONS (
    conn_url_id '<key_ID>'
  );
```

This command creates a connection (server) in PostgreSQL to access Redis using a Foreign Data Wrapper (FDW).

CREATE SERVER redis_server

- This creates a server definition in PostgreSQL named redis_server
- This server represents an external Redis database that PostgreSQL can connect to.

FOREIGN DATA WRAPPER redis_wrapper
This tells PostgreSQL that redis_server will use the Redis FDW (redis_wrapper) to communicate with Redis.

OPTIONS (conn_url_id '<key_ID>')

- Instead of storing the Redis connection URL directly, it **retrieves it securely** from the Vault.
- <key_ID> is the **ID** returned from this earlier query

5. Create a schema

create schema if not exists redis;

Why?
Organizes foreign tables separately and avoids clutter in your default public schema.

---

6. Create Foreign Tables

Example: Accessing a Redis Hash (like a key-value object):

```
create foreign table redis.memes (
  key text,
  value text
)
  server redis_server
  options (
    src_type 'hash',
    src_key 'memes'
  );
```

create foreign table redis.hash (key text, value text)

This defines a foreign table named hash inside the redis schema.
It has two columns:

key: the field representing the Redis hash field name.
value: the Redis hash field value.
Both are stored as text.

server redis_server
This tells PostgreSQL to connect to a foreign server named redis_server.

This server must have been defined earlier using CREATE SERVER.

```
options (
  src_type 'hash',
  src_key 'hash'
)
```

src_type 'hash': Specifies that the Redis data type is a hash (like a key-value object).

src_key 'hash': Points to the actual Redis key name, which is 'hash' in this case.

In Simple Terms:
"Hey Postgres, please create a table named redis.hash that reads data from a Redis hash whose key is 'hash'. Each row will have a key (the field name in the hash) and a value (the field's value). Use the Redis server called redis_server to get the data."

Example:

```
create foreign table redis.memes (
  key text,
  value text
)
  server redis_server
  options (
    src_type 'hash',
    src_key 'memes'
  );
```

7. Query Redis Data (Like SQL!)

```
SELECT * FROM redis.memes;
```

Or

```
SELECT value FROM redis.hash

WHERE key = '123' AND field =’dog’;
```

Remove FDW, server, and table if needed:

```
1. DROP FOREIGN TABLE IF EXISTS redis.user_profiles;
2. DROP SERVER IF EXISTS redis_server CASCADE;
3. DROP FOREIGN DATA WRAPPER IF EXISTS redis_wrapper CASCADE;
```

Steps to Get the Redis Connection URL from Upstash

Step 1: Create an Upstash Account

- Go to https://upstash.com.
- Click on "Start for free" and sign up using GitHub, Google, or email.

Step 2: Create a New Redis Database

- After logging in, go to the Upstash Console.
- Click on “Create Database”.
- Fill out the form:
    - Name: e.g. my-app-redis
    - Region: Choose a region close to your users (e.g., US-East or Europe)
    - Type: Select Global (accessible worldwide) or Single Region
- Click “Create”.

Step 3: View the Connection Details

- Once the database is created, you’ll be redirected to the database dashboard. Scroll down to the “Connect” section.

- **You will see connection details like:**
    - **REST API URL (for HTTP-based access)**
    - **Redis TLS URL (starts with rediss://)**
    - **Redis URL (non-TLS, starts with redis://)**

Step 4: Copy the Redis Connection URL

- Copy either the:
    - rediss://:<password>@<host>:<port> — ✅ recommended for security
    - or redis://:<password>@<host>:<port> — 🔧 use for local testing (non-TLS)

How to connect Upstash Redis in your code:

```
import { connect } from "https://deno.land/x/redis@v0.29.4/mod.ts";

const redis = await connect({
    hostname: "redis-19724.c262.us-east-1-3.ec2.redns.redis-cloud.com",
    port: 19724,
    password: "cu4Eg3dkCSgCTeTosurCWbGAkpVMXBpU", // If authentication is required
});
```

Steps to Get the Redis Connection URL from Redis Cloud:

Step 1: Create a Redis Cloud Account

1. Go to https://redis.io/cloud/
2. Sign up using Google, GitHub, or your email address
3. After signup, you'll be redirected to the Redis Cloud console

---

Step 2: Create a Redis Database

1. In the console, on the left side bar click on Databases , click on Create database
2. Choose:
    - Cloud provider (e.g., AWS, GCP, Azure)
    - Region (pick one near your users)
    - Database name (e.g., my-app-db)
    - Choose the free plan
    - Leave other options as default or configure as needed
3. Click “conform” to create the database

Step 3: View Connection Details

1. Once the database is created, click on it to open the details
2. Go to the "Endpoints" and click on "Connect" section
3. U will be able to see three options:
    - **Redis Insights**: A free graphical interface for analyzing and managing Redis data across all operating systems and deployments.

- **Redis CLI**: A command-line utility that allows you to send commands to Redis and read the server's replies directly from the terminal.

- **Redis Client**: Libraries provided for various programming languages to connect and interact with Redis databases within your application code.

4. Click on Redis CLI to get the connection url

5. You'll see:

    - Host (e.g., redis-12345.c10.us-east-1-2.ec2.cloud.redislabs.com)

    - Port (usually 6379)

    - Password (click to reveal it)

How to connect Redis Cloud in your code:

```
import { Redis } from 'https://deno.land/x/upstash_redis@v1.19.3/mod.ts'

 const redis = new Redis({

url:"https://talented-prawn-57335.upstash.io",

token:"Ad_3AAIjcDE0NDhkYmFkZGIzNGY0ODM5OGE0YmM2ZTg4Njg3MDI4YnAxMA"

 })
```